Implementation of Data Preprocessing

Aim

To implement **data preprocessing** techniques that convert raw, inconsistent data into a clean and suitable form for machine learning, by performing **data loading, handling missing values, removing duplicates, outlier treatment, encoding categorical variables, feature scaling** and **train-test splitting**, and to analyze the effect of preprocessing on the data quality and **model performance**.

Algorithm

A. Data Preprocessing Implementation

Input: Raw dataset **D** (rows = samples, columns = features and target)

Step 1: Load the dataset (for example, from a CSV file) and display its **shape, data types, first rows** and **summary statistics**.

Step 2: Handle missing values:

1. Detect the missing values in each column.
2. For **numerical** columns, fill with the **mean or median**.
3. For **categorical** columns, fill with the **mode**.
4. Drop a row or column if the proportion of missing values is too high.

Step 3: Remove duplicate records and correct inconsistent entries (for example, spelling or case differences).

Step 4: Detect and treat outliers using the **IQR method**:

1. Compute **Q1, Q3** and **IQR = Q3 − Q1**.
2. Mark values outside **[Q1 − 1.5 × IQR, Q3 + 1.5 × IQR]** as outliers.
3. **Remove or cap** (clip) these values.

Step 5: Encode categorical variables:

1. Use **Label Encoding** for ordinal features and the target.
2. Use **One-Hot Encoding** for nominal features.

Step 6: Separate the features **X** and the target **y**, then **split** the data into **training and test sets** (for example, 80:20).

Step 7: Feature scaling (fit on the training set only, then apply to both sets):

1. **Standardization: $z = (x - \mu) / \sigma$**
2. **Min-Max normalization: $x' = (x - x_{min}) / (x_{max} - x_{min})$**

Step 8: Output the cleaned and scaled training and test sets.

B. Performance Analysis Procedure

Step 1: Display the dataset **before and after** preprocessing (shape, missing-value count, duplicates).

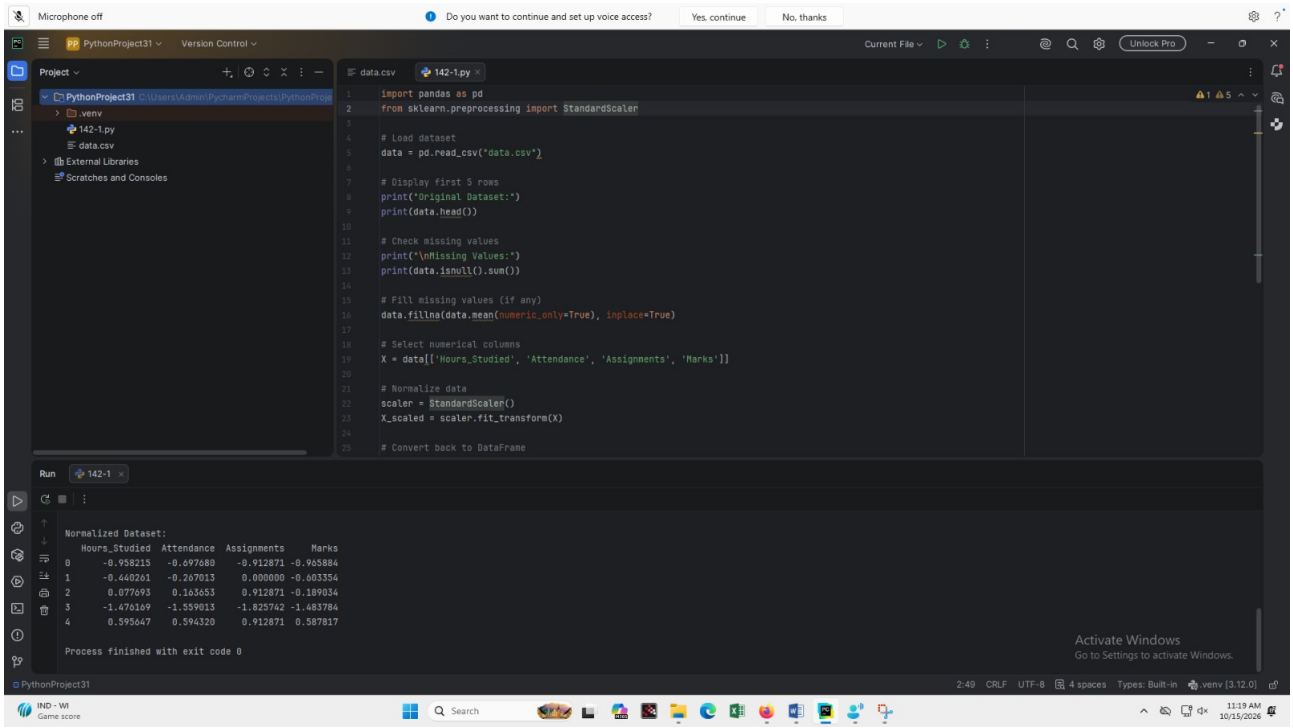
Step 2: Compare the **feature distributions** before and after scaling using **histograms or box plots**.

Step 3: Train a model (for example, **Logistic Regression** or **KNN**) on the **raw data** (baseline) and on the **preprocessed data**.

Step 4: Compute **accuracy, precision, recall and F1-score** for both models.

Step 5: Compare the results and **conclude** how preprocessing improved data quality and model performance.

Program


Microphone off
Do you want to continue and set up voice access?
Yes, continue
No, thanks
PythonProject31
Version Control
Current File
Unlock Pro
Project
PythonProject31
.venv
142-1.py
data.csv
External Libraries
Scratches and Consoles
data.csv
142-1.py
import pandas as pd
from sklearn.preprocessing import StandardScaler
Load dataset
data = pd.read_csv("data.csv")
Display first 5 rows
print("Original Dataset:")
print(data.head())
Check missing values
print("\nMissing Values:")
print(data.isnull().sum())
Fill missing values (if any)
data.fillna(data.mean(numeric_only=True), inplace=True)
Select numerical columns
X = data[['Hours_Studied', 'Attendance', 'Assignments', 'Marks']]
Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
Convert back to DataFrame
Run
142-1
Normalized Dataset:
Hours_Studied Attendance Assignments Marks
0 -0.958215 -0.697688 -0.912871 -0.965884
1 -0.440261 -0.267013 0.000000 -0.603354
2 0.077693 0.163653 0.912871 -0.189034
3 -1.476169 -1.559913 -1.825742 -1.483784
4 0.595647 0.594320 0.912871 0.587817
Process finished with exit code 0
Activate Windows
Go to Settings to activate Windows.
PythonProject31
2:49 CRLF UTF-8 4 spaces Types: Built-in .venv [3.12.0]
IND - WI
Game score
Search
11:19 AM
10/15/2026